

GRADIENT BOOSTING CLASSIFIER

=> Step 1 : Import Libraries

```
import pandas as pd
import numpy as np
```

=> Data Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns
```

=> Dataset Split

```
from sklearn.model_selection import train_test_split
```

=> Model

```
from sklearn.ensemble import GradientBoostingClassifier
```

=> Evaluation

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)
```

=> Cross Validation

```
from sklearn.model_selection import cross_val_score
```

=> Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV
```

=> Step 2 : Load Dataset

```
df = pd.read_csv("diabetes.csv")
```

```
print(df.head())
```

=> Step 3 : Basic Information

```
print(df.shape)
print(df.info())
print(df.describe())
```

=> Step 4 : Check Missing Values

```
print(df.isnull().sum())
```

=> Step 5 : Define Features and Target

```
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]
```

=> Step 6 : Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

=> Step 7 : Create Gradient Boosting Model

```
model = GradientBoostingClassifier(
```

=> Number of Trees

```
n_estimators=100,
```

=> Learning Speed

```
learning_rate=0.1,
```

=> Maximum Tree Depth

```
max_depth=3,
```

=> Minimum Samples to Split

```
min_samples_split=2,
```

=> Minimum Samples in Leaf

```
min_samples_leaf=1,
```

=> Percentage of Training Data Used

```
subsample=0.8,
```

=> Loss Function
 loss="log_loss",

=> Random Seed
 random_state=42

)

=>

=====

=====

=> Step 8 : Train Model

=>

=====

=====

model.fit(X_train, y_train)

=>

=====

=====

=> Step 9 : Prediction

y_pred = model.predict(X_test)

=> Probability Prediction

y_prob = model.predict_proba(X_test)[: ,1]

=> Step 10 : Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", accuracy)

=> Step 11 : Precision

precision = precision_score(y_test, y_pred)

print("Precision :", precision)

=> Step 12 : Recall

```
recall = recall_score(y_test, y_pred)

print("Recall :", recall)

=> Step 13 : F1 Score

f1 = f1_score(y_test, y_pred)

print("F1 Score :", f1)

=> Step 14 : ROC AUC Score

auc = roc_auc_score(y_test, y_prob)

print("ROC AUC :", auc)

=> Step 15 : Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)

sns.heatmap(

    cm,
    annot=True,
    fmt='d',
    cmap="Blues"

)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()

=> Step 16 : Classification Report

print(classification_report(y_test, y_pred))

=> Step 17 : Feature Importance
```

```
importance = pd.DataFrame({  
    "Feature":X.columns,  
    "Importance":model.feature_importances_  
})
```

```
importance = importance.sort_values(  
    by="Importance",  
    ascending=False  
)
```

```
print(importance)
```

=> Plot Feature Importance

```
plt.figure(figsize=(8,5))
```

```
plt.bar(  
    importance["Feature"],  
    importance["Importance"]  
)
```

```
plt.xticks(rotation=90)
```

```
plt.title("Feature Importance")
```

```
plt.show()
```

=> Step 18 : Cross Validation

```
cv = cross_val_score(  
    model,
```

```
X,  
y,  
cv=5,  
scoring="accuracy"  
)  
  
print("Cross Validation Scores :", cv)  
print("Average Accuracy :", cv.mean())  
  
=> Step 19 : Hyperparameter Tuning  
  
param_grid = {  
    "n_estimators":[50,100,200],  
    "learning_rate":[0.01,0.1,0.2],  
    "max_depth":[2,3,4],  
    "subsample":[0.8,1.0]  
}  
  
grid = GridSearchCV(  
    GradientBoostingClassifier(random_state=42),  
    param_grid=param_grid,  
    cv=5,  
    scoring="accuracy",  
    n_jobs=-1  
)
```

```
grid.fit(X_train, y_train)
print("Best Parameters :")
print(grid.best_params_)
print()
print("Best Accuracy :")
print(grid.best_score_)
```

=> Step 20 : Best Model

```
best_model = grid.best_estimator_
prediction = best_model.predict(X_test)
print("Final Accuracy :")
print(accuracy_score(y_test, prediction))
```